Deep Learning Program

Transforming How Your Teams Work With AI

The Deep Learning Program (DLP) is Improving's 12-week, cohort-based training engagement designed to move your engineering and product teams from occasional AI use to disciplined, repeatable AI-assisted delivery. Participants don't just learn prompting techniques — they build evaluated, version-controlled commands and workflows that live in your codebase long after the program ends.

Every team member leaves with artifacts they built on your real work, against your real backlog, in your real development environment.

© 2026 Improving

HOW THE PROGRAM WORKS

The DLP runs six 4-hour trainer-led sessions over 12 weeks, with an Improving Embedded Engineer present on your team between sessions — coaching in real sprints, not in a classroom. Sessions are spaced two weeks apart so participants have time to apply what they learned before the next session builds on it.

All roles attend every session. Product, Development, and Testing team members learn together because the skills they build are interdependent. A developer writing tests needs to understand how Acceptance Criteria was generated. A tester building a test plan needs to see the unit test coverage that came before it. The program is designed so that each role's work makes every other role's work better.

The program is organized into two loops:

- **Loop 1 (Sessions 1–3, Weeks 1–6):** Build the foundational chain — Acceptance Criteria, unit tests, and test plans — as evaluated, repeatable commands.
- **Loop 2 (Sessions 4–6, Weeks 7–12):** Deepen the discipline — measure what you built, graduate commands to autonomous skills, and connect everything into bounded agents.

© 2026 Improving

WHAT YOUR EMPLOYEES WILL LEARN

SESSION 1 — ACCEPTANCE CRITERIA GENERATION

Week 1 | Role Focus: Product & Process

The program starts here because Acceptance Criteria is the upstream artifact that determines quality everywhere downstream. Vague Acceptance Criteria produces vague tests and incomplete test plans. Precise Acceptance Criteria sets the whole delivery chain up to succeed.

In this session, every participant will:

- Configure an AI-assisted development environment and write their first project-level rules file
- Learn the RTCC prompt structure (Role, Task, Context, Constraints) — the framework used to build every command in the program
- Write evaluation criteria for what good Acceptance Criteria looks like *before* building any command, establishing the Evaluation-Driven Development (EDD) habit from day one
- Build a working Acceptance Criteria generation command using RTCC, run it against real backlog stories, and score the output against their own criteria
- Apply the Load-Bearing Rule: audit every instruction in their command against the criteria, and remove anything that doesn't correspond to a measurable outcome

What participants leave with: A committed, evaluated Acceptance Criteria generation command they can use with their Embedded Engineer the next morning — plus a baseline AI Adoption Maturity Assessment score that tracks their growth across the program.

SESSION 2 — UNIT TEST GENERATION

Week 3 | Role Focus: Development

By Week 3, participants have two weeks of real Acceptance Criteria generation behind them. This session uses that Acceptance Criteria as the specification for unit tests — making the dependency between the two explicit and visible to every role in the room.

In this session, every participant will:

- Understand the most common failure modes in AI-generated tests: false coverage, tautological assertions, happy-path-only output, and testing implementation details instead of behavior
- Write evaluation criteria for test output before building any command
- Build a RTCC-structured unit test generation command for their project's language and framework, grounded in their team's actual coding standards
- Generate tests from real stories using their Acceptance Criteria and evaluate each against their written criteria
- Participate in a cross-role discussion that makes the Acceptance Criteria quality dependency concrete: Product and Process participants see exactly how their upstream work shapes what developers can generate downstream

© 2026 Improving

What participants leave with: A committed test generation command with a load-bearing audit completed, tests generated from real stories with criteria scores documented, and a written hypothesis about the most common failure mode in their output.

SESSION 3 — TEST PLANNING

Week 5 | Role Focus: Testing

Session 3 closes Loop 1 by assembling the full chain for the first time. Participants generate a test plan from a real story's complete artifact set — Acceptance Criteria and unit tests — and see, for the first time, what the end-to-end chain they've been building actually produces.

In this session, every participant will:

- Understand what a test plan covers that unit tests don't: integration scenarios, end-to-end flows, risk-based coverage decisions, regression considerations, and non-functional concerns
- Write evaluation criteria for test plan output before building the command
- Build a RTCC-structured test plan command that takes the full upstream artifact set as input and produces coverage grounded in the Acceptance Criteria
- Walk through the complete Loop 1 chain on a real story — story → Acceptance Criteria → tests → test plan — and evaluate quality at every step
- Complete a Loop 1 retrospective: what changed since Week 1, and what would each participant change about their Session 1 command now that they've seen it in use for five weeks

What participants leave with: A committed test plan command, test plans generated from real sprint stories with criteria scores, a written Loop 1 retrospective, and a clear preview of what Loop 2 will add to what they've built.

SESSION 4 — EVALUATION-DRIVEN DEVELOPMENT (EDD)

Week 7 | Role Focus: Product & Process

After six weeks of using their commands in real sprints, most participants have been changing their commands without a systematic way to know whether the changes made the output better. That's not iteration — that's drift. Session 4 formalizes the Evaluation-Driven Development discipline and introduces Promptfoo, the tool that makes measurement fast enough to be habitual.

In this session, every participant will:

- Learn the EDD cycle — Red (baseline your command), Green (iterate until criteria pass), Refactor (remove instructions that pass no test) — and understand its precise parallel to test-driven development
- Configure Promptfoo against a command they own, writing 4–6 assertions that correspond to the evaluation criteria they wrote in Sessions 1–3
- Run a documented improvement cycle: form a specific hypothesis about why a criterion is failing, make one change, re-run the evaluator, and record the result

© 2026 Improving

- Apply the Refactor step: use Promptfoo to confirm which instructions in their command are actually load-bearing, and remove those that aren't
- Discuss what EDD reveals — including that AI output often varies more across inputs than across command versions, and that writing criteria before building reveals what you actually want from a command

What participants leave with: A Promptfoo configuration with a documented baseline, one committed improvement cycle with hypothesis and conclusion, a next hypothesis written and ready, and a leaner command after the Refactor step.

SESSION 5 — COMMANDS TO SKILLS

Week 9 | Role Focus: Development

For eight weeks, participants have worked with commands — prompt files they invoke explicitly, at a time they choose. Session 5 introduces the next step on the automation spectrum: skills, which the model can invoke autonomously when it recognizes the right context. The session draws a sharp, principled line between the two — because graduating a command to a skill without the right prerequisites is one of the most common ways good AI work becomes an invisible liability.

In this session, every participant will:

- Articulate the difference between a command and a skill, and understand why explicit invocation control matters before it is removed
- Apply the Team Consensus Criteria to evaluate whether a command is ready for skill graduation: exact inputs that should trigger it, exactly what it does when triggered, edge cases where it should not run, and how to verify it triggered correctly
- Write invocation test cases for their command — positive cases (the skill should invoke), negative cases (the skill should stay silent), and quality cases (given that it invoked, the output meets the evaluator criteria)
- Either graduate a command to a skill with test cases committed, or produce a specification document that defines exactly what's still needed before graduation is safe
- Understand the full automation spectrum — Commands → Skills → Hooks + Sentinel Files → Governed Workflows — and what each step requires before moving to the next

What participants leave with: Written Team Consensus Criteria for at least one command, a set of positive, negative, and quality invocation test cases, and either a graduated skill or a specification document for one that isn't yet ready.

SESSION 6 — AGENTS & WORKFLOWS

Week 11 | Role Focus: Testing

The final session shows what it looks like when commands and skills connect into something larger than any individual artifact. A bounded agent, in the DLP context, is a structured orchestration of commands and skills that handles a multi-step process within a coherent context. The Acceptance Criteria → tests → test plan chain that the program has been building toward is exactly that.

Session 6 also closes the program. Participants present their best work to a peer, complete their final AI Adoption Maturity Assessment, and make a specific, observable commitment for how their practice continues.

© 2026 Improving

In this session, every participant will:

- Understand what a bounded agent is, what it is not, and what it requires to build safely
- Learn context window discipline: how output quality degrades in later steps of a long chain, and how to design agents that stay inside a reliable window
- Build or specify a bounded agent for a real process they own, using commands and skills already built and evaluated during the program
- Evaluate the agent across the full chain — scoring each step independently and identifying the weakest link and its root cause
- Present their most-used command, their evaluator, and their skill or agent to a peer — articulating not just what they built, but the evaluation and iteration discipline behind it
- Complete the final AI Adoption Maturity Assessment and compare it to their Week 1 and Week 7 scores
- State a specific, observable commitment for their next sprint: not "I'll use AI more," but a named behavior change tied to a concrete artifact or outcome

What participants leave with: A committed bounded agent or agent specification, a chain-level evaluator with step-by-step scoring, a completed final maturity assessment, and a written commitment statement that the Embedded Engineer will follow up on in Week 12.

© 2026 Improving

WHAT THE PROGRAM PRODUCES

By the end of 12 weeks, every participant has built and committed to your codebase:

- A working Acceptance Criteria generation command, evaluated and load-bearing-audited
- A unit test generation command tuned to your project's language, framework, and standards
- A test plan generation command grounded in your team's actual testing practices
- A Promptfoo evaluation configuration with documented baselines and iteration history
- A skill or skill specification for at least one graduated command
- A bounded agent or agent specification for a real multi-step workflow

These are not training exercises. They are real artifacts, built on real backlog items, designed to be used by real teams in the sprint that follows each session. The Embedded Engineer's job is to make sure they are.

© 2026 Improving

THE MEASURE OF SUCCESS

The program tracks progress using the AI Adoption Maturity Assessment — a 21-question, 7-dimension instrument administered at Week 1, Week 7, and Week 11. The dimensions cover:

- Prompt Engineering & AI Interaction
- Task Agents & Single-Box Automation
- Workflow Orchestration & Coordination
- Output Evaluation & Quality Judgment
- Governance, Trust & Safety
- Trust & Working Identity
- Team & Organizational Practices

DLP Completion — the program's outcome target — is defined by four measurable criteria: iterative refinement of commands, skills, and workflows; deliberate use of evaluation frameworks; at least one workflow step operating at 95%+ reliability; and changed team behavior to use that high-reliability command instead of performing the task manually.

The program ends not with a certificate, but with a team that owns evaluated, iterated artifacts — and knows exactly what the path forward looks like.

© 2026 Improving

READY TO LEARN MORE?

Contact Improving to discuss how the Deep Learning Program can be structured for your team's size, technology stack, and delivery context.

improving.com

© 2026 Improving